MAGENTO 2 · PLUGIN (INTERCEPTOR) DEMO

Codilar_CartPlugins

How the three cart plugins work, a file-by-file code explanation, and the questions to expect when presenting this demo.

Module	`app/code/Codilar/CartPlugins`
Platform	Magento Open Source 2.4.9 · PHP 8.5
Plugged class	`Magento\Checkout\Model\Cart`
Plugin types shown	**before** **around** **after**
Scope	Storefront only (`etc/frontend/di.xml`)
Date	October 2026

1. What the demo does
2. Plugin fundamentals
3. Request flow
4. Module structure
5. File-by-file code explanation
6. Tested scenarios
7. Install, configure and test
8. Design decisions and pitfalls
9. Known limitations
10. Expected questions and answers

1. What the demo does

The module changes how the storefront shopping cart behaves **without editing any core file**. It hooks three rules onto two public methods of `Magento\Checkout\Model\Cart`, using each of the three plugin types once.

#	Rule	Plugin type	Method intercepted	Result for the shopper
1	Block guest add-to-cart for certain products	**before**	`Cart::addProduct()`	A guest adding a product flagged *Login Required to Buy* sees **"Please log in to buy this product."** Nothing is added.
2	Free gift with expensive phones	**after**	`Cart::save()`	When the cart holds a phone (Phones category) priced above $1000, the *iPhone 13 Pro Silicone Case* is added at **$0.00** with "A free phone cover has been added to your cart." It is removed again if the phone leaves the cart.
3	Block duplicate items	**around**	`Cart::addProduct()`	Adding a product that is already in the cart is skipped and shows **"This item is already in your cart."**

Demo data used

- **Phones** category: Apple iPhone 13 Pro ($1099.99), Samsung Galaxy S10 ($699.99), Oppo F19 Pro Plus 5G ($399.99), Vivo X21 ($499.99), Realme XT ($349.99).
- **Phone Cases** category: one case per phone, $14.99–$29.99.
- *Login Required to Buy* = Yes on **Samsung Galaxy S10** and **Vivo X21**.
- Only the **iPhone 13 Pro** is above the $1000 threshold, so it is the phone that earns the gift.

> **Nothing on the admin side was coded.** No admin templates, layouts, UI components, menus or configuration were changed. The only admin-visible change is the new *Login Required to Buy* Yes/No field on the product edit page, which Magento displays automatically because the module registers it as a product attribute.

2. Plugin fundamentals

A **plugin** (also called an *interceptor*) is a class that changes the behaviour of a **public method** of another class by running code before, after, or around it. Magento makes this possible by generating an `Interceptor` subclass of the target class and handing that subclass out everywhere the original is requested.

The three plugin types

Type	Method signature	Runs	Can do
before	`beforeName($subject, ...$args)`	Before the original method.	Inspect or change the arguments. Return `null` to keep them, or an array of new arguments. Can throw to stop the call.
around	`aroundName($subject, callable $proceed, ...$args)`	Instead of the original method; it decides whether and when to call `$proceed()`.	Run code on both sides, change arguments and result, or skip the original entirely by not calling `$proceed()`.
after	`afterName($subject, $result, ...$args)`	After the original method returns.	Inspect or replace the return value. Must return the (possibly changed) result. Since 2.2 it also receives the original arguments.

What the generated interceptor looks like

This is the real generated code for `addProduct()` in `generated/code/Magento/Checkout/Model/Cart/Interceptor.php`. If any plugins are registered for the method, the call is routed through `___callPlugins()`; otherwise the parent method is called directly.

generated/code/Magento/Checkout/Model/Cart/Interceptor.php (excerpt)

```
class Interceptor extends \Magento\Checkout\Model\Cart implements
\Magento\Framework\Interception\InterceptorInterface
{
    public function addProduct($productInfo, $requestInfo = null)
    {
        $pluginInfo = $this->pluginList->getNext($this->subjectType, 'addProduct');
        return $pluginInfo ? $this->___callPlugins('addProduct', func_get_args(), $pluginInfo) :
parent::addProduct($productInfo, $requestInfo);
    }
}
```

Execution order and `sortOrder`

When several plugins target the same method, Magento sorts them by `sortOrder` (lowest first) and runs them like nested layers:

1. befores and the first half of arounds of the lowest `sortOrder` plugin,
2. then the next plugin's before / around-first-half, and so on,
3. the original method,

4. then the around-second-halves and afters in reverse order.

In this module, `addProduct()` has two plugins: **GuestAddToCartBlock** (sortOrder 10, before) and **DuplicateItemBlock** (sortOrder 20, around). So the guest check always runs first; the duplicate check runs only if the guest check let the call through. **FreeGift** (sortOrder 30) is on a different method (`save()`), so its sortOrder does not interact with the other two.

Where plugins cannot be used

- `final` methods and `final` classes
- non-public (`private` / `protected`) methods, and `static` methods
- `__construct()` and other magic methods such as `__get`
- virtual types
- objects created before the interception framework starts (e.g. some bootstrap objects)

Plugin vs observer vs preference

Technique	Use it when
Plugin	You need to change the input, output or execution of one specific public method. Several plugins can coexist on the same method.
Observer	Magento already dispatches an event at the right moment and you only need to react to it (e.g. `catalog_controller_category_init_after` to change a category before its page is built).
Preference	You must replace a whole class (e.g. swap `Magento\Theme\Block\Html\Footer` for your own subclass). Only one preference can win, so it is the most conflict-prone option.

3. Request flow

The diagram shows what happens when a shopper clicks **Add to Cart** on the storefront (Magento 2.4.9). The plugin layers are coloured by type.

POST /checkout/cart/add
Magento\Checkout\Controller\Cart\Add::execute() loads the product and calls the AddProductToCart service

▼

Magento\Checkout\Model\AddProductToCart::execute()
calls `$cart->addProduct($product, $buyRequest)`, then `$cart->save()`

▼ addProduct()

① GuestAddToCartBlock::beforeAddProduct() — sortOrder 10
Guest + product flagged *Login Required to Buy* → throws *"Please log in to buy this product."* Otherwise returns `null` (arguments unchanged).

▼

③ DuplicateItemBlock::aroundAddProduct() — sortOrder 20
Product already a (non-gift) line in the quote → throws *"This item is already in your cart."* and `$proceed()` is never called. Otherwise calls `$proceed()`.

▼ $proceed()

Original Cart::addProduct()
adds the product to the quote

▼ save()

Original Cart::save()
collectTotals() → quoteRepository->save() → checkout_cart_save_after event

▼

② FreeGift::afterSave() — sortOrder 30
Phone > $1000 in cart and no gift → add case at $0 and save again. Gift present but no qualifying phone → remove it and save again. Otherwise do nothing.

▼

Success
Controller adds "You added X to your shopping cart."

Exception thrown by ① or ③
Controller's `catch (LocalizedException $e)` shows `$e->getMessage()` as an error and redirects back. `save()` never runs.

Other actions that reach the plugins

Storefront action	Plugins that run
Add to cart (product page, category listing)	① guest check, ③ duplicate check, then ② free gift
Update quantities on the cart page (`Cart\UpdatePost`)	② free gift only (`updateItems()` + `save()`; `addProduct()` is not used)
Remove an item on the cart page (`Cart\Delete`)	② free gift only (`removeItem()` + `save()`)
Remove an item from the mini-cart (`Model\Sidebar`)	② free gift only
Admin order creation, REST and GraphQL carts	None — they do not use `Magento\Checkout\Model\Cart`, and the plugins are registered for the storefront area only.

4. Module structure

app/code/Codilar/CartPlugins/

```
CartPlugins/
├── registration.php                        registers the module with Magento
├── etc/
│   ├── module.xml                          module name + load order
│   └── frontend/
│       └── di.xml                          plugin declarations + FreeGift settings (storefront only)
├── Plugin/
│   └── Checkout/
│       ├── GuestAddToCartBlock.php         ① before  Cart::addProduct
│       ├── DuplicateItemBlock.php          ③ around  Cart::addProduct
│       └── FreeGift.php                    ② after   Cart::save
└── Setup/
    └── Patch/
        └── Data/
            └── AddLoginRequiredAttribute.php creates the "Login Required to Buy" attribute
```

File	Responsibility
`registration.php`	Tells Magento that a module named `Codilar_CartPlugins` lives in this folder.
`etc/module.xml`	Declares the module and makes it load after Catalog, Checkout and Customer.
`etc/frontend/di.xml`	Registers the three plugins on `Cart`, passes settings to FreeGift, injects a session proxy into GuestAddToCartBlock.
`Plugin/Checkout/GuestAddToCartBlock.php`	Rule 1 – guest block.
`Plugin/Checkout/DuplicateItemBlock.php`	Rule 3 – duplicate block.
`Plugin/Checkout/FreeGift.php`	Rule 2 – free gift.
`Setup/Patch/Data/AddLoginRequiredAttribute.php`	One-time data patch that creates the product attribute used by rule 1.

5. File-by-file code explanation

5.1 registration.php

app/code/Codilar/CartPlugins/registration.php

```
1 <?php
2 use Magento\Framework\Component\ComponentRegistrar;
3
4 ComponentRegistrar::register(ComponentRegistrar::MODULE, 'Codilar_CartPlugins', __DIR__);
```

- **Line 4** – `ComponentRegistrar::register()` records that the component of type `MODULE` named `Codilar_CartPlugins` is located in `__DIR__` (this folder).
- Composer's autoloader loads every `registration.php` under `app/code` at bootstrap, so this is how Magento discovers the module.
- The name must be `Vendor_Module` and match `etc/module.xml`.

5.2 etc/module.xml

app/code/Codilar/CartPlugins/etc/module.xml

```
1  <?xml version="1.0"?>
2  <config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3          xsi:noNamespaceSchemaLocation="urn:magento:framework:Module/etc/module.xsd">
4      <module name="Codilar_CartPlugins">
5          <sequence>
6              <module name="Magento_Catalog"/>
7              <module name="Magento_Checkout"/>
8              <module name="Magento_Customer"/>
9          </sequence>
10     </module>
11 </config>
```

- **Line 4** – the module name. After `bin/magento module:enable` it is listed in `app/etc/config.php`.
- **Lines 5–9** – `<sequence>` controls **load order**, not a hard dependency. Our module is loaded after Catalog (product attribute APIs), Checkout (the `Cart` class we plug into) and Customer (the session). That way our configuration is merged after theirs and the data patch runs after the catalog tables exist.
- No `setup_version` is needed: schema and data are handled by declarative schema and patches in Magento 2.3+.

5.3 etc/frontend/di.xml

app/code/Codilar/CartPlugins/etc/frontend/di.xml

```
1  <?xml version="1.0"?>
2  <config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3          xsi:noNamespaceSchemaLocation="urn:magento:framework:ObjectManager/etc/config.xsd">
4      <!-- All three plugins wrap Magento\Checkout\Model\Cart, the model the storefront
5           add-to-cart, update and delete controllers go through.
6           Declared in etc/frontend so admin order creation is not affected.
7           sortOrder: the guest check (before, 10) runs before the duplicate check (around, 20). -->
8      <type name="Magento\Checkout\Model\Cart">
9          <plugin name="codilar_cartplugins_guest_block"
10                 type="Codilar\CartPlugins\Plugin\Checkout\GuestAddToCartBlock"
11                 sortOrder="10"/>
12         <plugin name="codilar_cartplugins_duplicate_block"
13                 type="Codilar\CartPlugins\Plugin\Checkout\DuplicateItemBlock"
14                 sortOrder="20"/>
```

```
        <plugin name="codilar_cartplugins_free_gift"
                type="Codilar\CartPlugins\Plugin\Checkout\FreeGift"
                sortOrder="30"/>
    </type>

    <!-- Which phones earn a gift, and which product is given away. -->
    <type name="Codilar\CartPlugins\Plugin\Checkout\FreeGift">
        <arguments>
            <argument name="phonesCategoryUrlKey" xsi:type="string">phones</argument>
            <argument name="priceThreshold" xsi:type="number">1000</argument>
            <argument name="giftSku" xsi:type="string">case-iphone-13-pro</argument>
        </arguments>
    </type>

    <!-- Session is not needed until a plugin actually runs. -->
    <type name="Codilar\CartPlugins\Plugin\Checkout\GuestAddToCartBlock">
        <arguments>
            <argument name="customerSession"
xsi:type="object">Magento\Customer\Model\Session\Proxy</argument>
        </arguments>
    </type>
</config>
```

- **Location** – the file is in `etc/frontend/`, so the configuration applies only to the storefront area. Admin, cron, REST and GraphQL do not load it.
- **Lines 8–18** – `<type name="Magento\Checkout\Model\Cart">` picks the class to intercept. Each `<plugin>` has:
  - `name` – a unique id; another module can use the same name with `disabled="true"` to switch the plugin off.
  - `type` – the plugin class.
  - `sortOrder` – execution order (10 → guest check first, 20 → duplicate check second).

  The method names (`beforeAddProduct`, `aroundAddProduct`, `afterSave`) inside the classes decide which methods are intercepted and how. The XML only links the classes to `Cart`.
- **Lines 21–27** – constructor arguments for `FreeGift`: the category URL key that defines a "phone", the price threshold (1000) and the SKU given away. Changing them needs only this file plus a cache flush, with no PHP changes.
- **Lines 30–34** – injects `Magento\Customer\Model\Session\Proxy` instead of the real session. The proxy is a lightweight stand-in, generated by Magento, that only creates the real session the first time it is used. Session classes are expensive and have side effects, so proxies are the recommended way to inject them.

5.4 Setup/Patch/Data/AddLoginRequiredAttribute.php

app/code/Codilar/CartPlugins/Setup/Patch/Data/AddLoginRequiredAttribute.php

```
<?php
declare(strict_types=1);

namespace Codilar\CartPlugins\Setup\Patch\Data;

use Magento\Catalog\Model\Product;
use Magento\Catalog\Model\ResourceModel\Eav\Attribute;
use Magento\Eav\Model\Entity\Attribute\Source\Boolean;
use Magento\Eav\Setup\EavSetupFactory;
use Magento\Framework\Setup\ModuleDataSetupInterface;
use Magento\Framework\Setup\Patch\DataPatchInterface;

/**
 * Adds the "Login Required to Buy" Yes/No product attribute.
```

```php
 * Products with it set to Yes cannot be added to the cart by guests
 * (see Plugin\Checkout\GuestAddToCartBlock).
 */
class AddLoginRequiredAttribute implements DataPatchInterface
{
    public const ATTRIBUTE_CODE = 'login_required_to_buy';

    public function __construct(
        private readonly ModuleDataSetupInterface $moduleDataSetup,
        private readonly EavSetupFactory $eavSetupFactory
    ) {
    }

    public function apply(): self
    {
        $eavSetup = $this->eavSetupFactory->create(['setup' => $this->moduleDataSetup]);
        $eavSetup->addAttribute(Product::ENTITY, self::ATTRIBUTE_CODE, [
            'type' => 'int',
            'label' => 'Login Required to Buy',
            'input' => 'boolean',
            'source' => Boolean::class,
            'default' => 0,
            'global' => Attribute::SCOPE_GLOBAL,
            'required' => false,
            'user_defined' => true,
            'visible' => true,
            'used_in_product_listing' => true,
            'group' => 'General',
        ]);

        return $this;
    }

    public static function getDependencies(): array
    {
        return [];
    }

    public function getAliases(): array
    {
        return [];
    }
}
```

- **Implements** `DataPatchInterface` – Magento runs each data patch once, during `bin/magento setup:upgrade`, and records it in the `patch_list` table so it never runs twice.
- **Line 20** – `ATTRIBUTE_CODE` is a public constant, so the plugin reads the same code (`login_required_to_buy`) and the two can never get out of sync.
- **Lines 22–26** – constructor property promotion (PHP 8) with `readonly`. `ModuleDataSetupInterface` provides the DB connection; `EavSetupFactory` creates an `EavSetup` helper.
- **Lines 28–46** – `apply()` calls `addAttribute()` on the product entity:
  - `type => int`, `input => boolean`, `source => Boolean`: a Yes/No field stored as 0/1 in `catalog_product_entity_int`.
  - `global => SCOPE_GLOBAL`: one value for all store views.
  - `user_defined`, `visible`, `group => General`: shows on the admin product form in the General section and can be edited or deleted like any custom attribute. **This is why the field appears in the admin without any admin code.**
  - `used_in_product_listing`: loaded on listing collections too, so the flag is available there if needed.

- **Lines 48–56** – `getDependencies()` (patches that must run first) and `getAliases()` (old names of this patch) are required by the interface. Both are empty here.

5.5 Plugin/Checkout/GuestAddToCartBlock.php `before`

app/code/Codilar/CartPlugins/Plugin/Checkout/GuestAddToCartBlock.php

```php
<?php
declare(strict_types=1);

namespace Codilar\CartPlugins\Plugin\Checkout;

use Codilar\CartPlugins\Setup\Patch\Data\AddLoginRequiredAttribute;
use Magento\Catalog\Api\ProductRepositoryInterface;
use Magento\Catalog\Model\Product;
use Magento\Checkout\Model\Cart;
use Magento\Customer\Model\Session;
use Magento\Framework\Exception\LocalizedException;

/**
 * BEFORE plugin on Cart::addProduct().
 * Runs before the product is added. If a guest tries to buy a product flagged
 * "Login Required to Buy", throwing stops addProduct() from running at all; the
 * add-to-cart controller catches the exception and shows the message.
 */
class GuestAddToCartBlock
{
    public function __construct(
        private readonly Session $customerSession,
        private readonly ProductRepositoryInterface $productRepository
    ) {
    }

    /**
     * @param int|Product $productInfo
     * @param mixed $requestInfo
     * @throws LocalizedException
     */
    public function beforeAddProduct(Cart $subject, $productInfo, $requestInfo = null): null
    {
        if ($this->customerSession->isLoggedIn()) {
            return null;
        }

        $product = $productInfo instanceof Product
            ? $productInfo
            : $this->productRepository->getById((int)$productInfo);

        if ($product->getData(AddLoginRequiredAttribute::ATTRIBUTE_CODE)) {
            throw new LocalizedException(__('Please log in to buy this product.'));
        }

        // null = leave the original arguments unchanged.
        return null;
    }
}
```

- **Lines 21–25** – dependencies: the customer session (actually the proxy from di.xml) and the product repository.
- **Line 32 – the signature.** `before` + `AddProduct` = intercept `addProduct()` before it runs. The first parameter is always the intercepted object (`$subject`); the rest mirror the original method's parameters (`$productInfo`, `$requestInfo`).
- **Lines 34–36** – logged-in customers are never blocked; return immediately.

- **Lines 38–40** – `Cart::addProduct()` accepts either a product object or a product ID, so the plugin handles both. The storefront controller passes a fully loaded product, so the repository is only a fallback.
- **Lines 42–44** – if the flag is set, throw `LocalizedException`. Because a before plugin runs first, throwing stops the whole chain: the duplicate plugin, the original `addProduct()` and `save()` never run. The controller catches the exception and shows the message as an error.
- **Line 47** – returning `null` from a before plugin means “keep the original arguments”. Returning an array would replace them (e.g. to change the requested quantity).

5.6 Plugin/Checkout/DuplicateItemBlock.php around

app/code/Codilar/CartPlugins/Plugin/Checkout/DuplicateItemBlock.php

```php
<?php
declare(strict_types=1);

namespace Codilar\CartPlugins\Plugin\Checkout;

use Magento\Catalog\Model\Product;
use Magento\Checkout\Model\Cart;
use Magento\Framework\Exception\LocalizedException;

/**
 * AROUND plugin on Cart::addProduct().
 * Decides whether the original method runs at all: if the product is already in the
 * cart, $proceed() is never called. Throwing (rather than just returning) also stops the
 * controller from showing "You added ... to your shopping cart".
 * Quantity can still be changed on the cart page, which does not use addProduct().
 */
class DuplicateItemBlock
{
    /**
     * @param int|Product $productInfo
     * @param mixed $requestInfo
     * @throws LocalizedException
     */
    public function aroundAddProduct(Cart $subject, callable $proceed, $productInfo, $requestInfo =
null): Cart
    {
        $productId = $productInfo instanceof Product ? (int)$productInfo->getId() : (int)$productInfo;

        foreach ($subject->getQuote()->getAllVisibleItems() as $item) {
            // The free gift line does not count, so the customer can still buy that case.
            if ($item->getOptionByCode(FreeGift::OPTION_CODE)) {
                continue;
            }
            if ((int)$item->getProductId() === $productId) {
                throw new LocalizedException(__('This item is already in your cart.'));
            }
        }

        return $proceed($productInfo, $requestInfo);
    }
}
```

- **Line 24 – the signature.** An around plugin receives `$subject`, then `callable $proceed` (the next plugin in the chain, or the original method), then the original arguments. It must return what the original returns (`Cart`).
- **Line 26** – normalise the input to a product ID (object or ID, as in the guest plugin).
- **Line 28** – `getAllVisibleItems()` returns the cart lines the shopper sees (parent items only, no configurable children).

- **Lines 29–32** – the free gift line is skipped. Without this, a shopper who received the case for free could not also buy a paid copy. The gift line is recognised by its custom option (see FreeGift).
- **Lines 33–35** – if the product is already in the cart, throw. `$proceed()` is never reached, so the original `addProduct()` does not run. We throw instead of simply `return $subject;` because a silent return would make the controller show “You added X to your shopping cart”, which would be wrong.
- **Line 38** – otherwise call `$proceed()` with the same arguments and return its result. Forgetting this line would silently break add-to-cart for every product: the most common around-plugin mistake.

5.7 Plugin/Checkout/FreeGift.php after

app/code/Codilar/CartPlugins/Plugin/Checkout/FreeGift.php

```php
<?php
declare(strict_types=1);

namespace Codilar\CartPlugins\Plugin\Checkout;

use Magento\Catalog\Api\ProductRepositoryInterface;
use Magento\Catalog\Model\ResourceModel\Category\CollectionFactory as CategoryCollectionFactory;
use Magento\Checkout\Model\Cart;
use Magento\Framework\Exception\NoSuchEntityException;
use Magento\Framework\Message\ManagerInterface;
use Magento\Quote\Model\Quote\Item;

/**
 * AFTER plugin on Cart::save().
 * Runs once the cart has been saved (after add, update and delete). If the cart holds a
 * phone priced above the threshold, a free phone cover is added at price 0; if not, any
 * gift already in the cart is removed. The cart is then saved once more to store the change.
 */
class FreeGift
{
    /** Custom option that marks the gift line and keeps it separate from a paid copy of the same case.
*/
    public const OPTION_CODE = 'codilar_free_gift';

    /** Stops the second save() below from running this plugin again. */
    private bool $isProcessing = false;

    private ?int $phonesCategoryId = null;

    public function __construct(
        private readonly ProductRepositoryInterface $productRepository,
        private readonly CategoryCollectionFactory $categoryCollectionFactory,
        private readonly ManagerInterface $messageManager,
        private readonly string $phonesCategoryUrlKey,
        private readonly float $priceThreshold,
        private readonly string $giftSku
    ) {
    }

    public function afterSave(Cart $subject, Cart $result): Cart
    {
        if ($this->isProcessing) {
            return $result;
        }

        $quote = $subject->getQuote();
        $giftItem = null;
        $qualifies = false;
        foreach ($quote->getAllVisibleItems() as $item) {
            if ($item->getOptionByCode(self::OPTION_CODE)) {
```

```
                $giftItem = $item;
            } elseif ($this->isQualifyingPhone($item)) {
                $qualifies = true;
            }
        }

        // Nothing to change: gift present and earned, or absent and not earned (qty kept at 1).
        if ($qualifies === ($giftItem !== null) && (!$giftItem || (float)$giftItem->getQty() === 1.0))
{
            return $result;
        }

        $this->isProcessing = true;
        try {
            if ($qualifies && $giftItem) {
                $giftItem->setQty(1);
            } elseif ($qualifies) {
                if (!$this->addGift($quote)) {
                    return $result;
                }
                $this->messageManager->addSuccessMessage(__('A free phone cover has been added to your
cart.'));
            } else {
                $quote->removeItem($giftItem->getId());
            }
            // The first save already collected totals and each address cached its item list,
            // so reset both or the gift line keeps its catalog price.
            $quote->setTotalsCollectedFlag(false);
            foreach ($quote->getAllAddresses() as $address) {
                $address->unsetData('cached_items_all');
            }
            $subject->save();
        } finally {
            $this->isProcessing = false;
        }

        return $result;
    }

    private function isQualifyingPhone(Item $item): bool
    {
        $product = $item->getProduct();

        return (float)$product->getFinalPrice() > $this->priceThreshold
            && in_array($this->getPhonesCategoryId(), array_map('intval', $product->getCategoryIds()),
true);
    }

    private function addGift(\Magento\Quote\Model\Quote $quote): bool
    {
        try {
            // Clone so the custom option does not stick to the repository's cached product.
            $gift = clone $this->productRepository->get($this->giftSku, false, $quote->getStoreId());
        } catch (NoSuchEntityException) {
            return false;
        }
        if (!$gift->isSalable()) {
            return false;
        }

        $gift->addCustomOption(self::OPTION_CODE, '1');
        $item = $quote->addProduct($gift, 1);
        if (is_string($item)) {
            // addProduct() returns an error string when the product cannot be added.
            return false;
        }
        $item->setCustomPrice(0);
        $item->setOriginalCustomPrice(0);
```

```
115
116         return true;
117     }
118
119     private function getPhonesCategoryId(): int
120     {
121         if ($this->phonesCategoryId === null) {
122             $this->phonesCategoryId = (int)$this->categoryCollectionFactory->create()
123                 ->addAttributeToFilter('url_key', $this->phonesCategoryUrlKey)
124                 ->setPageSize(1)
125                 ->getFirstItem()
126                 ->getId();
127         }
128
129         return $this->phonesCategoryId;
130     }
131 }
```

Properties and constructor (lines 21–37)

- `OPTION_CODE` – name of the custom option stored on the gift's quote item (`quote_item_option` table). It marks the gift line, is used by DuplicateItemBlock to skip it, and keeps it separate from a paid copy of the same case.
- `$isProcessing` – recursion guard. `afterSave()` itself calls `save()`, which would trigger `afterSave()` again, endlessly.
- `$phonesCategoryId` – memoised category ID, so the category is looked up once per request.
- Constructor: three services plus the three scalar settings from di.xml (`$phonesCategoryUrlKey`, `$priceThreshold`, `$giftSku`). Magento's object manager matches the di.xml `<argument name="…">` to these parameter names.

`afterSave()` (lines 39–85)

- **Signature** – `afterSave(Cart $subject, Cart $result)`: the subject, then the original return value. An after plugin must return the result, here unchanged.
- **Lines 41–43** – if we are inside our own second save, do nothing.
- **Lines 45–54** – one pass over the visible cart lines: remember the gift line (if any) and whether any line is a qualifying phone.
- **Lines 56–59** – nothing to do when the state is already correct: (gift earned and present with qty 1) or (not earned and absent). This early exit keeps normal cart saves cheap.
- **Lines 61–72** – three outcomes:
  - earned and present but qty changed → reset qty to 1 (a shopper cannot get 5 free cases);
  - earned but missing → `addGift()` and show the success message; if the gift can't be added (unknown SKU, out of stock), stop quietly;
  - not earned but present → remove the gift line.
- **Lines 73–78** – before saving again, reset the quote's *totals collected* flag and each address's cached item list (`cached_items_all`). The first `save()` already calculated totals; without the reset, the second save skips recalculation and the gift shows its catalog price ($29.99) instead of $0.00. This bug was found during testing (see section 8).
- **Line 79** – `$subject->save()` reuses core's save logic (collect totals, persist quote, update session, fire events).
- **Lines 80–82** – `finally` clears the guard even if an exception happens.

`isQualifyingPhone()` (lines 87–93)

- Qualifies when the product's **final price** (so special prices count) is above the threshold **and** the product is assigned to the Phones category.
- `getCategoryIds()` loads the product's category IDs on demand; they are cast to int for a strict comparison.

`addGift()` (lines 95–117)

- **Lines 97–102** – load the gift by SKU for the quote's store. It is **cloned** because the product repository caches instances: adding a custom option to the cached object would leak the gift marker onto any later use of the same product in the request.
- **Lines 103–105** – don't give away a disabled or out-of-stock product.
- **Line 107** – `addCustomOption()` attaches the `codilar_free_gift` marker. When Magento decides whether an added product can merge into an existing line, it compares custom options (`Quote\Item::representProduct()`). The marker therefore keeps the gift and a paid copy as two separate lines.
- **Lines 108–112** – `Quote::addProduct()` returns an error string instead of throwing when it fails, so that case is checked.
- **Lines 113–114** – `setCustomPrice(0)` and `setOriginalCustomPrice(0)` override the price for this line only. Core's admin custom-price code (`Quote\Item\Updater::setCustomPrice()`) sets the same two fields.

`getPhonesCategoryId()` (lines 119–130)

- Resolves the URL key `phones` to a category ID once and caches it in the property. Using the URL key instead of a hard-coded ID keeps the module portable between environments, where IDs differ.

6. Tested scenarios

All scenarios were run end-to-end over HTTP against the local storefront (`http://localhost:8080`) as a real browser would: form key, cookies and the add-to-cart controller. Messages were read from the `mage-messages` cookie, which is how Magento delivers them to the browser.

#	Shopper	Action	Result	Rule
1	Guest	Add Samsung Galaxy S10 (flagged)	“Please log in to buy this product.” Cart unchanged.	①
2	Guest	Add Realme XT	“You added Realme XT to your shopping cart.”	–
3	Guest	Add Realme XT again	“This item is already in your cart.” Qty stays 1.	③
4	Customer	Add Samsung Galaxy S10 (flagged)	Added – logged-in customers are not blocked.	①
5	Customer	Add Apple iPhone 13 Pro ($1099.99)	Added, plus “A free phone cover has been added to your cart.” Case line shows **$0.00**.	②
6	Customer	Add the same case as a paid item	Added as a separate $29.99 line; the $0.00 gift line remains.	②③
7	Customer	Remove the iPhone 13 Pro	Gift line removed automatically; paid case stays.	②
8	Customer	Add the iPhone 13 Pro again	Added, and the gift comes back at $0.00.	②
9	Customer	Add the paid case again	“This item is already in your cart.”	③

7. Install, configure and test

Install

```
terminal — from the Magento root
```

```
php bin/magento module:enable Codilar_CartPlugins
php bin/magento setup:upgrade          # registers the module and runs the data patch
php bin/magento setup:di:compile       # production mode only — regenerates interceptors
php bin/magento cache:flush
```

Configure

- **Which products guests cannot buy** – Admin → Catalog → Products → edit product → **Login Required to Buy** = Yes → Save.
- **Free gift settings** – edit the three `<argument>` values in `etc/frontend/di.xml` (category URL key, price threshold, gift SKU), then run `bin/magento cache:flush` (and `setup:di:compile` in production).
- **Switch one plugin off** without deleting code – in any later-loading module's `di.xml`: `<plugin name="codilar_cartplugins_free_gift" disabled="true"/>` inside the same `<type>`.

Manual test script

1. As a guest, open Samsung Galaxy S10 and click Add to Cart → login message.
2. As a guest, add Realme XT twice → second attempt shows the duplicate message.
3. Log in, add Apple iPhone 13 Pro → the cart shows the iPhone plus the plum case at $0.00.
4. Change the gift's quantity to 3 and click Update → it returns to 1.
5. Delete the iPhone → the gift disappears.

Useful checks

```
terminal
```

```
php bin/magento module:status Codilar_CartPlugins
php bin/magento dev:di:info "Magento\Checkout\Model\Cart"     # lists plugins per area
grep -n addProduct generated/code/Magento/Checkout/Model/Cart/Interceptor.php
```

8. Design decisions and pitfalls

Throwing `LocalizedException` instead of returning early

The add-to-cart controller shows a success message after `addProduct()` returns. A plugin that silently skips the add would still trigger “You added X to your shopping cart”. Throwing a `LocalizedException` sends control to the controller's `catch` block, which displays our message as an error and redirects back. Only `LocalizedException` messages are shown to the shopper; any other exception becomes a generic error.

Before for the guest check, around for the duplicate check

The guest check only needs to look at the input and either allow or stop the call, so a before plugin is the simplest fit. The duplicate check is written as an around plugin to demonstrate the type, including deciding whether `$proceed()` is called. Because it throws, a before plugin would give the same result. In production code, prefer before/after over around whenever possible: around plugins add stack depth and make it easy to forget `$proceed()`.

Free gift on `save()`, not on `addProduct()`

`save()` runs after add, quantity update and delete, so one plugin can both add and remove the gift and correct its quantity. An after plugin on `addProduct()` would never see deletes.

Recursion guard

Calling `$subject->save()` inside `afterSave()` goes through the interceptor again. The `$isProcessing` flag (cleared in `finally`) makes the nested call a no-op.

> **Bug found during testing – stale totals.** At first the gift appeared at $29.99 and only dropped to $0.00 on the next cart change. `Cart::save()` calls `Quote::collectTotals()`, which sets `totals_collected_flag` and does nothing on later calls. In addition, each quote address caches its item list in `cached_items_all`, so the totals collectors did not see the newly added gift. The fix (FreeGift.php, lines 73–78) resets both before the second save.

Cloning the gift product

`ProductRepository` keeps loaded products in an in-memory cache. Adding the gift marker to the cached object would also mark the product for any other code in the same request, for example if the shopper adds the same case themselves.

Storefront scope and session proxy

Registering the plugins in `etc/frontend/di.xml` keeps admin order creation free of storefront rules. The customer session is injected as a `Proxy` so the session is only started when the guest check actually runs.

9. Known limitations

- **Configurable products** – the duplicate check compares product IDs. Two different sizes of the same configurable product share the parent ID, so the second size would be blocked. The current catalogue has only simple products.
- **APIs** – REST and GraphQL carts (and therefore headless/PWA storefronts) do not use `Magento\Checkout\Model\Cart`, so none of the rules apply there. Enforcing them everywhere would mean plugging into `Magento\Quote\Model\Quote::addProduct()` or using quote events instead.
- **The gift cannot be refused** – if the shopper deletes the gift while the phone is still in the cart, the delete's `save()` adds it straight back.
- **One gift** – the gift SKU is fixed (the iPhone 13 Pro case) whatever phone qualifies. A per-phone gift would need a mapping, e.g. a product attribute holding the gift SKU.
- **Settings in di.xml** – threshold, category and gift SKU are developer settings. An admin page under Stores → Configuration (`system.xml`) would let merchants change them.
- **Gift message** – the success message is added during save, so it can also appear after a quantity update that re-adds the gift. This is intended.

10. Expected questions and answers

Plugin basics

Q1. What is a plugin in Magento 2?

A class that changes the behaviour of a public method of another class by running code before, after or around it, without editing or replacing that class. Magento generates an `Interceptor` subclass that calls the plugins and hands it out in place of the original class.

Q2. What are the three plugin types and when would you use each?

before – validate or change the arguments; **after** – inspect or change the return value or react once the method succeeded; **around** – when you must control whether or when the original runs. Prefer before/after; use around only when the other two can't do the job.

Q3. What must a before plugin return?

`null` to keep the original arguments, or an array with the new arguments in the original order. In this demo, GuestAddToCartBlock returns `null`.

Q4. What does an after plugin receive and return?

It receives `$subject`, the original method's `$result`, and (since 2.2) the original arguments. It must return a result: either the same one or a modified one. FreeGift returns `$result` unchanged.

Q5. What happens if an around plugin never calls `$proceed()`?

The original method and every plugin with a higher sortOrder on that method are skipped. DuplicateItemBlock uses this on purpose for duplicates. Doing it by accident is a common bug that silently disables the method.

Q6. Why are around plugins discouraged?

They add extra stack frames on every call and make the code harder to follow. If `$proceed()` is forgotten or called with wrong arguments, the original method breaks silently. Magento's coding standard recommends before/after unless you need to wrap or skip the call.

Q7. Which methods cannot be plugged?

Final methods or final classes, non-public methods, static methods, `__construct` and other magic methods, virtual types, and objects created before interception is initialised.

Q8. How does `sortOrder` work?

Plugins on the same method run from the lowest sortOrder to the highest on the way in (befores and the first half of arounds), then in reverse on the way out (second half of arounds and afters). Here the guest check (10) runs before the duplicate check (20).

Q9. Where are the interceptors generated, and when do you need `setup:di:compile`?

In `generated/code/<Vendor>/<Module>/.../Interceptor.php`. In developer mode they are generated on demand. In production mode you must run `setup:di:compile` after adding or changing plugins.

Q10. What is the difference between `etc/di.xml` and `etc/frontend/di.xml`?

`etc/di.xml` is global and applies to every area. Files in `etc/frontend`, `etc/adminhtml`, `etc/webapi_rest` and so on apply only to that area. This demo uses `frontend` so admin order creation is unaffected.

Q11. How do you disable another module's plugin?

Declare a plugin with the same `name` on the same `type` with `disabled="true"` in a module that loads later (use `<sequence>`).

Q12. Plugin, observer or preference: how do you choose?

Use an observer if a suitable event exists and you only need to react. Use a plugin to change one public method's input, output or execution. Use a preference only to replace an entire class, because only one preference can win.

About this module

Q13. Why plug into `Magento\Checkout\Model\Cart` and not the controller?

All storefront add/update/delete actions go through this model, so one plugin covers the product page, the category listing, the cart page and the mini-cart. A controller plugin would cover only one entry point.

Q14. How does the shopper see the message from a plugin?

The plugin throws `LocalizedException`. The add-to-cart controller catches it, adds the text as an error with the message manager, and redirects back. Magento sends it to the browser in the `mage-messages` cookie.

Q15. Why throw instead of just returning from the plugin?

If the plugin returns normally, the controller thinks the add succeeded and shows “You added X to your shopping cart”. Throwing both stops the add and shows our message instead.

Q16. How does the guest plugin know which products are restricted?

It reads the `login_required_to_buy` product attribute created by the data patch. Admins set it per product, so no code change is needed to restrict another product.

Q17. Did you change any admin code to show the new field?

No. The data patch registers a user-defined product attribute in the General group, and Magento's product form renders all such attributes automatically.

Q18. Why is the session injected as a Proxy?

Constructing `Customer\Model\Session` starts session handling and loads data. A proxy delays that until a method is called, so creating the plugin stays cheap. Magento recommends proxies for sessions and other heavy dependencies.

Q19. Could the duplicate check be a before plugin?

Yes. Since it throws, a before plugin would behave the same. It is an around plugin here to demonstrate that type, specifically the ability to skip `$proceed()`.

Q20. Why is the free gift an after plugin on `save()` and not on `addProduct()`?

`save()` runs after adds, quantity updates and deletes. That lets one plugin add the gift, remove it when the phone leaves, and reset its quantity. After `save()`, the cart contents are also final.

Q21. How do you stop `afterSave()` from calling itself forever?

A private `$isProcessing` flag is set before the nested `save()` and cleared in `finally`. While it is set, `afterSave()` returns immediately.

Q22. How is the gift line kept separate from a paid copy of the same case?

The gift carries a custom option `codilar_free_gift`. `Quote\Item::representProduct()` compares custom options before merging, so a product without the option becomes a new line. The duplicate plugin also skips lines with this option.

Q23. How is the gift made free?

`setCustomPrice(0)` and `setOriginalCustomPrice(0)` on the quote item: the same mechanism as the admin's custom price. Totals then use 0 for that line.

Q24. Why clone the product from the repository?

The repository caches product objects for the request. Adding the gift option to the cached object would affect other code that loads the same product later in the request.

Q25. The gift first showed $29.99. What was the cause?

The first `save()` had already collected totals and set `totals_collected_flag`. The address had also cached its item list without the gift. The second save therefore skipped price calculation. Resetting the flag and `cached_items_all` before re-saving fixed it.

Q26. How do you change the threshold or the gift product?

Edit the `<argument>` values for `FreeGift` in `etc/frontend/di.xml` and flush the cache. The object manager injects them into the constructor by parameter name.

Q27. Why find the Phones category by URL key instead of ID?

Entity IDs differ between environments (local, staging, production). URL keys are chosen by people and stay the same.

Q28. What happens if the gift product is deleted or out of stock?

`addGift()` catches `NoSuchEntityException` and checks `isSalable()`. If the gift can't be given, the phone is still added normally and no gift or message appears.

Q29. Can a shopper get several free cases by changing the quantity?

No. On the next save the plugin sees a gift quantity other than 1 and resets it.

Q30. Do these rules apply to admin orders or the REST/GraphQL APIs?

No. The plugins are registered for the storefront area only, and the APIs use quote services rather than `Checkout\Model\Cart`. To enforce the rules everywhere, plug into `Quote::addProduct()` in the global `etc/di.xml`.

Q31. Does the duplicate check stop the shopper from increasing quantity?

No. It only blocks a second add-to-cart. Changing quantity on the cart page uses `updateItems()`, which is not intercepted.

Q32. What would break with configurable products?

All variants share the parent product ID. A second size or colour of the same configurable product would be treated as a duplicate. Comparing the selected child product or the buy request options would fix this.

Q33. How was the module tested?

End-to-end over HTTP against the local storefront as a guest and as a logged-in customer (section 6). Each step's message was read from the `mage-messages` cookie, and the cart page was parsed for lines, prices and quantities.

Q34. How can you confirm which plugins are active on a class?

`bin/magento dev:di:info "Magento\Checkout\Model\Cart"` lists the plugins. The generated `Interceptor.php` shows which methods are routed through `___callPlugins()`.